

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №5

Выполнил:

Шалунов Андрей

Группа К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Выделить самостоятельные модули в вашем приложении;

Провести разделение своего API на микросервисы (минимум, их должно быть 3);

Настроить сетевое взаимодействие между микросервисами.

Ход работы

1. Анализ существующего монолита

В ходе анализа сервиса аренды недвижимости были выделены независимые предметные области, которые слабо связаны между собой по данным и могут развиваться отдельно. К таким доменам отнесены: аутентификация, профили пользователей, объекты недвижимости, фотографии объектов, бронирования и обмен сообщениями.

На основании анализа было принято решение реализовать микросервисную архитектуру, в которой каждый домен обслуживается отдельным сервисом.

2. Разделение API на микросервисы

В результате декомпозиции API было разделено на набор микросервисов (больше минимально требуемых трёх):

1. Auth-service

Отвечает за регистрацию пользователей, вход в систему и выдачу JWT. Содержит собственное хранилище данных для аутентификации (email и хеш пароля). После успешной регистрации выполняется сетевой вызов в user-service для создания профиля пользователя.

2. User-service

Отвечает за хранение и управление профилями пользователей (имя, email, телефон, пароль). Реализует получение данных текущего пользователя и обновление профиля. Для предотвращения ошибок валидации номера телефона применена единая стратегия валидации

(например, `IsPhoneNumber('RU')`), чтобы одинаковое правило действовало при регистрации и редактировании профиля.

3. Property-service

Реализует CRUD для объектов недвижимости, а также поиск и фильтрацию по параметрам (тип, цена, город, владелец). При создании объекта сервис проверяет существование владельца через сетевой вызов в `user-service`.

4. Photo-service

Отвечает за привязку фотографий к объектам недвижимости и хранение ссылок (URL) и описаний. При добавлении фотографии выполняется проверка существования объекта через запрос к `property-service`.

5. Booking-service

Управляет бронированиями объектов, хранит информацию о периоде аренды, стоимости и статусе сделки. При создании бронирования сервис выполняет сетевые запросы в `user-service` и `property-service` для получения данных и при необходимости сохраняет их локально.

6. Message-service

Реализует отправку сообщений между пользователями и хранение истории чатов. При создании сообщения сервис получает данные отправителя и получателя через `user-service`. Для корректной сортировки чатов по активности дополнительно обновляется поле `updated_at` у сущности `Chat` при каждом новом сообщении.

Таким образом, ответственность каждого микросервиса была ограничена конкретным доменом, что соответствует принципам микросервисной архитектуры.

3. Организация хранения данных

Для каждого микросервиса была выделена отдельная база данных Postgres. Работа с данными реализована через TypeORM, сущности описаны через декораторы @Entity. Входящие данные проверяются с помощью DTO и class-validator, что снижает вероятность некорректных запросов на уровне API.

4. Реализация API Gateway

Для обеспечения единой точки входа был реализован API Gateway на базе Express и express-http-proxy. Gateway выполняет маршрутизацию запросов на соответствующие сервисы по префиксам:

1. /api/auth → auth-service
2. /api/users → user-service
3. /api/properties → property-service
4. /api/photos → photo-service
5. /api/bookings → booking-service
6. /api/messages → message-service

Gateway пробрасывает заголовок Authorization в целевой сервис, благодаря чему все защищенные эндпоинты могут самостоятельно проверять JWT. В ходе настройки было обеспечено корректное проксирование полного пути (/api/...), чтобы избежать ошибок 404 из-за неправильного формирования URL на стороне gateway.

5. Настройка сетевого взаимодействия между сервисами

Сетевое взаимодействие между микросервисами выполнено через HTTP-вызовы (axios) внутри docker-сети по именам сервисов. Были реализованы следующие сценарии:

1. Auth-service → User-service: создание профиля пользователя после успешной регистрации.

2. Property-service → User-service: проверка владельца объекта недвижимости при создании/обновлении.
3. Photo-service → Property-service: проверка существования объекта перед добавлением фотографии.
4. Booking-service → User-service и Property-service: получение данных пользователя и объекта для создания бронирования.
5. Message-service → User-service: получение данных отправителя и получателя, создание чата и сохранение сообщения.

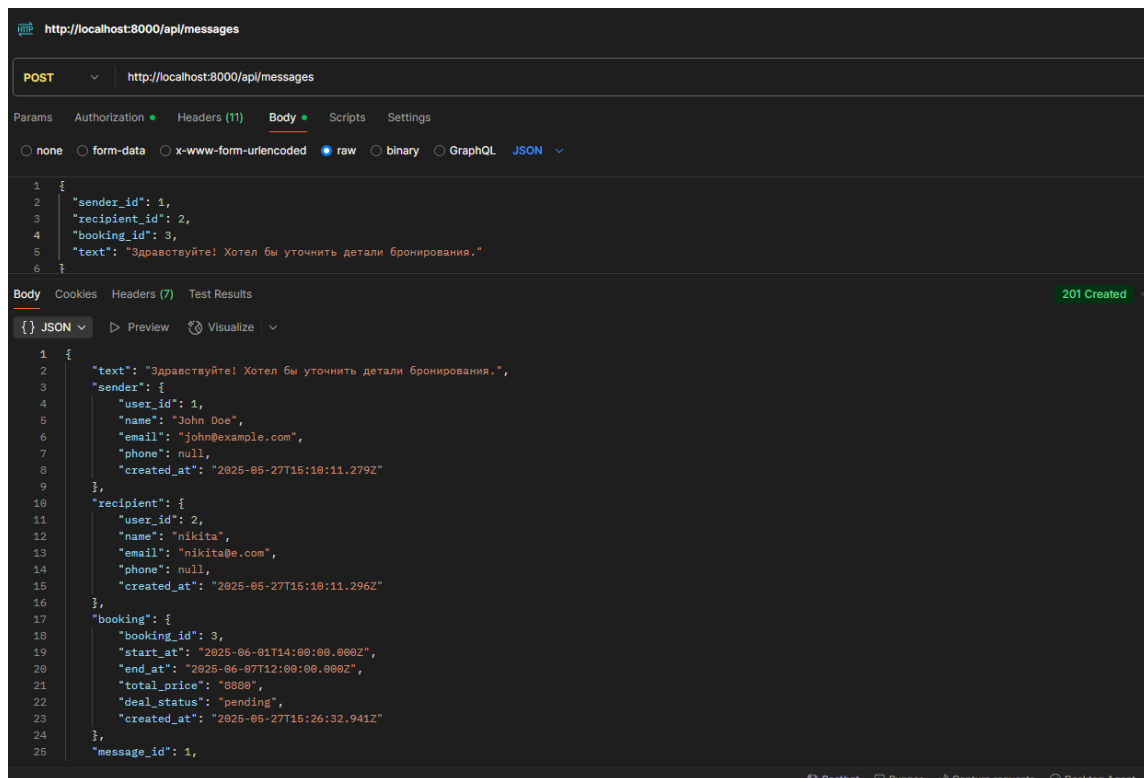
6. Тестирование

Проверка работоспособности выполнялась через Postman и фронтенд с обращением к единой точке входа Gateway <http://localhost:8000>

Были проверены основные сценарии:

1. регистрация и логин с получением JWT (/api/auth/register, /api/auth/login)
2. получение и обновление профиля (/api/users/me, /api/users/:id)
3. CRUD объектов недвижимости и фильтрация (/api/properties)
4. добавление и получение фотографий (/api/photos, /api/photos/by-property/:propertyId)
5. создание и получение бронирований (/api/bookings)
6. отправка сообщений и получение чатов (/api/messages, /api/messages/chats/user/:userId, /api/messages/chats/:chatId/messages)

Пример создания сообщения между пользователями:



Результаты тестирования подтвердили, что сервисы доступны через gateway, авторизация работает единообразно, а межсервисные вызовы выполняются корректно.

Вывод

В результате выполнения лабораторной работы исходное API было успешно мигрировано на микросервисную архитектуру. Были выделены самостоятельные домены и реализованы отдельные микросервисы, каждый со своей базой данных и API. Для объединения внешнего доступа был разработан API Gateway, который маршрутизирует запросы и пробрасывает заголовки авторизации.